Lecture 3 : Faire parler la station

BLE, GATT, fragmentation et protocoles de l'IoT industriel, projet EcoTri 4.0

Comment la XIAO transmet-elle une image à la Portenta par un lien de 20 octets ?

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi communiquer ?** : *Qui parle à qui dans la station, et que coûte un octet transmis ?*
- **BLE : radio et GAP** : *Comment deux cartes se trouvent-elles et partagent-elles le temps radio ?*
- **GATT et ATT** : *Comment organiser les données en services et caractéristiques ?*
- **Fragmenter une image** : *Comment faire passer 9 Ko dans des paquets de 20 à 244 octets, sans rien perdre ?*
- **Protocoles et énergie** : *BLE, Wi-Fi, MQTT ou OPC UA : lequel, où, et à quel coût énergétique ?*

Objectifs

Ce que vous saurez faire en sortant de la salle

À la fin de cette lecture, vous serez capables de :

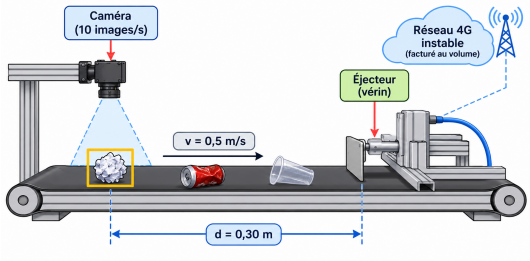
- **Décrire** la pile BLE et le rôle de chaque couche : PHY, liaison, L2CAP, ATT, GATT, GAP, SMP.
- **Concevoir** un service GATT : UUID, propriétés (read, write, notify), descripteur CCCD.
- **Calculer** le débit utile d'un lien BLE à partir du MTU, du PHY et de l'intervalle de connexion.
- **Mettre en œuvre** une fragmentation indexée avec contrôle d'intégrité (CRC) et retransmission sélective.
- **Comparer** BLE, Wi-Fi, MQTT et OPC UA, et **justifier** le principe « envoyer la décision, pas l'image » par un bilan énergétique.

Le fil rouge et le livrable

- La station EcoTri 4.0 sait **voir** (lecture 2) ; il faut maintenant qu'elle **parle** : XIAO → Portenta → opérateur.
- En fin de lecture, votre équipe rend le **jalon J3** : un **protocole documenté** (UUID, format des trames, format JSON des messages) et ses mesures de débit.

Le problème déclencheur

Un lien de 20 octets, une image de 9216 octets



La situation

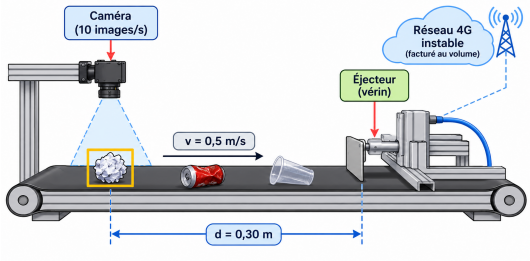
- La XIAO filme l'objet en 96×96 niveaux de gris : **9216 octets**.
- Par défaut, une notification BLE transporte **20 octets** (MTU de 23).
- Pour la mise au point et la collecte du dataset (lecture 5), la Portenta doit **recevoir l'image entière**.

À vous (5 min, par équipe)

1. Combien de paquets faut-il pour une image ?
2. Comment la Portenta sait-elle dans quel ordre les remettre, et s'il en manque un ?
3. Combien de temps cela prend-il si l'on envoie un paquet toutes les 15 ms ?

Le problème déclencheur

Un lien de 20 octets, une image de 9216 octets



La situation

- La XIAO filme l'objet en 96×96 niveaux de gris : **9216 octets**.
- Par défaut, une notification BLE transporte **20 octets** (MTU de 23).
- Pour la mise au point et la collecte du dataset (lecture 5), la Portenta doit **recevoir l'image entière**.

À vous (5 min, par équipe)

1. Combien de paquets faut-il pour une image ?
2. Comment la Portenta sait-elle dans quel ordre les remettre, et s'il en manque un ?
3. Combien de temps cela prend-il si l'on envoie un paquet toutes les 15 ms ?

Réponse

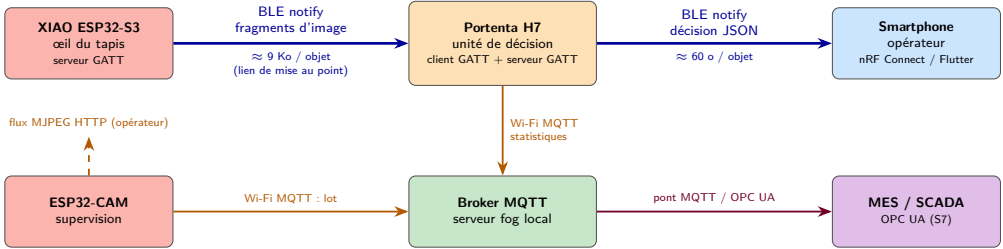
$\lceil 9216/20 \rceil = 461$ paquets sans en-tête. À un paquet par 15 ms : **6,9 s par image** ! Il faudra numéroter, agrandir le MTU et enchaîner plusieurs paquets par événement.

Pourquoi communiquer ?

Qui parle à qui dans la station, et que coûte un octet transmis ?

Pourquoi communiquer ?

Les liens de la station EcoTri 4.0



Règle de conception : plus on monte dans l'architecture, plus le message est petit et sémantique (pixels → étiquette → indicateur).

Lecture : deux liens BLE sur batterie (image de mise au point, décision vers l'opérateur), deux liens Wi-Fi alimentés vers le broker, un pont OPC UA vers le système d'information.

Pourquoi communiquer ?

Trois natures de flux, trois exigences

Flux	Exemple EcoTri	Exigence dominante	Choix naturel
Commande / événement	« objet métal, éjecter »	latence faible, fiabilité	BLE notify, MQTT QoS 1
Donnée volumineuse	image pour le dataset	débit, intégrité	BLE + fragmentation, Wi-Fi
État / indicateur	nombre d'objets, OEE	interopérabilité, historique	MQTT retain, OPC UA



on ne choisit pas « un » protocole pour la station : on choisit un protocole **par lien**, en fonction de la nature du flux, du débit et de l'énergie disponible.

- Énergie par bit transmis : $E_b = \dfrac{P_{\text{radio}}}{D_{\text{utile}}}$ (J/bit).
- Exemple ESP32-S3 en BLE : $P \approx 0{,}33$ W, $D \approx 500$ kbit/s $\Rightarrow$ $E_b \approx$ **0,66 μJ/bit**.
- À comparer à une MAC INT8 sur MCU : quelques **pJ** à quelques dizaines de pJ (Horowitz, Stanford, 2014) : un facteur $\mathbf{10^4}$ à $\mathbf{10^5}$.
- S'y ajoutent des **coûts fixes** : réveil de la radio, publicité, maintien de la connexion.

Conséquence (Warden & Situnayake ; Harvard CS249r)

Il est presque toujours plus économique de **calculer davantage pour transmettre moins**. C'est la justification énergétique du TinyML.

Mais attention

Le calcul n'est pas gratuit : sur un ESP32, une inférence de 60 ms coûte autant qu'environ 3 Ko envoyés en BLE. Le bilan se **mesure** (section 5).

Question flash

?

Un message de décision JSON fait 60 octets, une image 96×96 en gris 9216 octets. Avec $E_b = 0{,}66\ \mu$J/bit, quelle énergie radio pour chacun ? Quel rapport ?

Question flash

? Un message de décision JSON fait 60 octets, une image 96×96 en gris 9216 octets. Avec $E_b = 0{,}66\ \mu$J/bit, quelle énergie radio pour chacun ? Quel rapport ?

Réponse

Décision : $60 \times 8 \times 0{,}66\ \mu\text{J} \approx$ **0,32 mJ**. Image : $9216 \times 8 \times 0{,}66\ \mu\text{J} \approx$ **48,7 mJ**. Rapport $\approx$ **150**. Envoyer la décision plutôt que l'image divise l'énergie radio par deux ordres de grandeur, et libère le canal pour d'autres nœuds.

BLE : radio et GAP

Comment deux cartes se trouvent-elles et partagent-elles le temps radio ?

BLE : radio et GAP

Bluetooth Low Energy en une diapositive

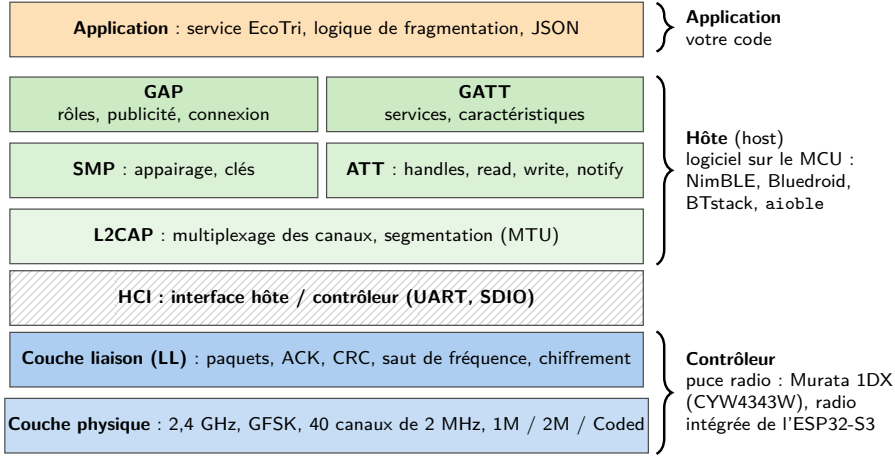
- Introduit en 2010 (Bluetooth 4.0, ex-Wibree de Nokia), conçu pour des **petits volumes** et des **piles bouton**.
- Radio 2,4 GHz, modulation GFSK, **40 canaux** de 2 MHz.
- **Asymétrique** : le périphérique dort le plus possible ; le central (téléphone, passerelle) porte la complexité.
- BLE 5 (2016) : PHY **2M** (débit $\times$ 2), PHY **Coded** (portée $\times$ 4), publicité étendue.

PHY	Débit radio	Usage
LE 1M	1 Mbit/s	par défaut, obligatoire
LE 2M	2 Mbit/s	débit, faible énergie/bit
Coded S=2	500 kbit/s	portée moyenne
Coded S=8	125 kbit/s	longue portée

! Bluetooth « classique » (BR/EDR : audio, port série) et BLE sont deux radios **différentes**, non compatibles entre elles.

BLE : radio et GAP

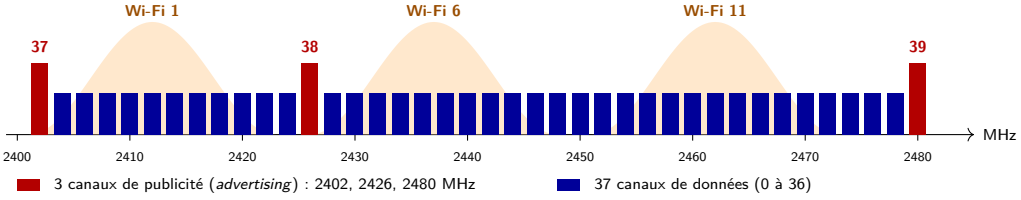
La pile protocolaire



d'après Bluetooth Core Specification v5.4, vol. 1, partie A ; Townsend et al., Getting Started with BLE, O'Reilly

BLE : radio et GAP

Couche physique : 40 canaux dans la bande ISM



Les canaux de publicité sont placés entre les canaux Wi-Fi 1, 6 et 11 les plus utilisés ; en connexion, le saut de fréquence évite les canaux brouillés.

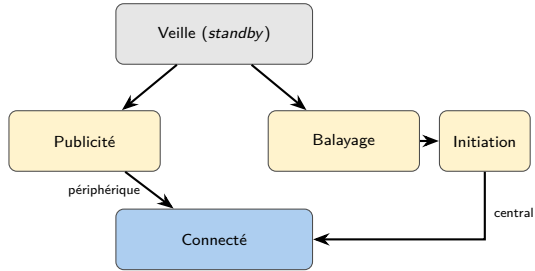
- En connexion, les deux nœuds **sautent** de canal à chaque événement (CSA #1 ou #2) : robustesse face au Wi-Fi et aux autres BLE.
- Le central peut exclure les canaux brouillés via la **carte des canaux** (*channel map*).

Quatre rôles GAP

- **Broadcaster** : publie sans accepter de connexion (balise).
- **Observer** : écoute les publicités sans se connecter.
- **Peripheral** : publie et **accepte** une connexion (XIAO, capteur).
- **Central** : balaie et **initie** la connexion (Portenta, smartphone).

Dans EcoTri

La Portenta est **centrale** face à la XIAO et **périphérique** face au smartphone : un nœud BLE peut jouer plusieurs rôles à la fois.



depuis Core 5.3, la spécification dit *central* / *peripheral* au lieu de *master* / *slave*.

BLE : radio et GAP

La publicité (advertising)

- Le périphérique émet un paquet de publicité sur les canaux 37, 38, 39, tous les **20 ms à 10,24 s**, plus un retard aléatoire de 0 à 10 ms (évite les collisions répétées).
- Charge utile : **31 octets** au format *AD structures* : [longueur | type | données].
- Types usuels : 0x01 drapeaux, 0x09 nom complet, 0x07 UUID 128 bits, 0xFF données fabricant.
- Le **balayage actif** demande 31 octets de plus (SCAN_RSP).

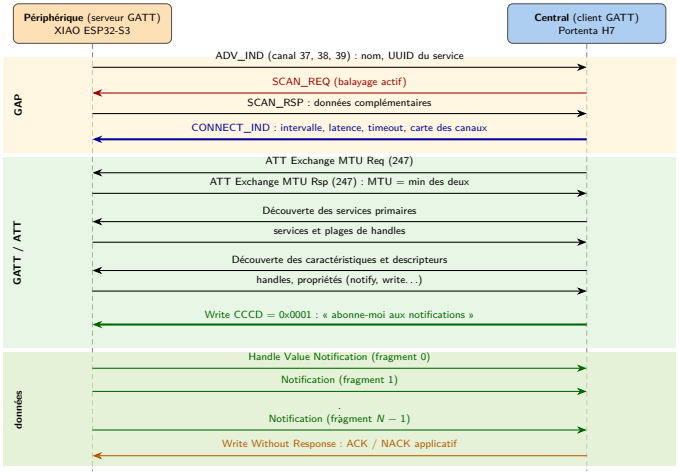
Compromis de l'intervalle de publicité

Court (20 ms) : découverte rapide, mais le courant moyen monte. Long (1 s) : quelques μA, mais l'opérateur attend.

> **!** les 31 octets suffisent pour une **décision** ! Une balise qui publie « dernière classe + compteur » dans les données fabricant se passe même de connexion (rôle broadcaster).

BLE : radio et GAP

De la publicité à la notification : la séquence complète

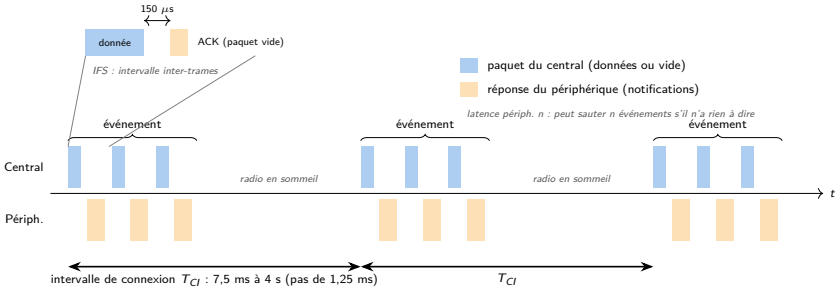


- **GAP** : le périphérique se signale, le central choisit et fixe les paramètres de connexion.
- **ATT/GATT** : négocier le MTU, découvrir les handles, puis s'abonner (CCCD).
- **Données** : les notifications s'enchaînent sans attendre de réponse.

> **!** la mise en place prend quelques intervalles de connexion ($\approx$ 100–300 ms) : on la fait **une fois**, puis on garde le lien.

BLE : radio et GAP

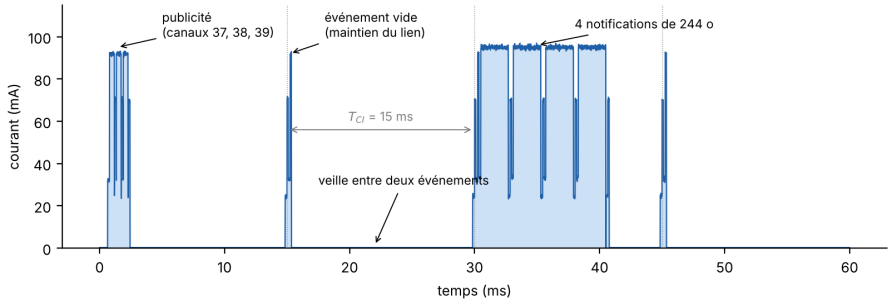
Les paramètres de connexion



- **Intervalle** T_{CI} : de 7,5 ms à 4 s. Fixe la **latence** et une partie du débit.
- **Latence du périphérique** n : nombre d'événements qu'il peut ignorer pour dormir.
- **Timeout de supervision** : 100 ms à 32 s sans échange $\Rightarrow$ lien déclaré perdu. Contrainte : $T_{sup} > (1 + n) \times 2\, T_{CI}$.
- Le **central décide** ; le périphérique ne peut que demander.

BLE : radio et GAP

À quoi ressemble la consommation ?



Lecture : le courant moyen est presque entièrement fixé par la **durée radio active** : $\overline{I} \approx \frac{1}{T_{CI}} \sum_k I_k t_k + I_{\text{veille}}$. Doubler T_{CI} divise presque par deux le coût du maintien de la connexion ; enchaîner les notifications dans un même événement amortit le réveil (profil modélisé d'après les fiches Espressif et les mesures de Siekkinen et al., Aalto, 2012).

Question flash

?

Le smartphone de l'opérateur doit recevoir une alerte en moins de 100 ms. Le périphérique a $T_{CI} = 50$ ms et une latence $n = 4$. L'exigence est-elle tenue ? Que proposez-vous ?

Question flash

? Le smartphone de l'opérateur doit recevoir une alerte en moins de 100 ms. Le périphérique a $T_{CI} = 50$ ms et une latence $n = 4$. L'exigence est-elle tenue ? Que proposez-vous ?

Réponse

Avec $n = 4$, le périphérique peut ne se réveiller que toutes les $(1 + 4) \times 50 = 250$ ms. Mais la latence ne s'applique qu'au **silence** : un périphérique qui a une donnée à envoyer se réveille au prochain événement, soit au pire **50 ms**. L'exigence tient côté envoi ; c'est le sens central → périphérique (commandes) qui subirait les 250 ms.

GATT et ATT

Comment organiser les données en services et caractéristiques ?

GATT et ATT

Un modèle client/serveur... inversé

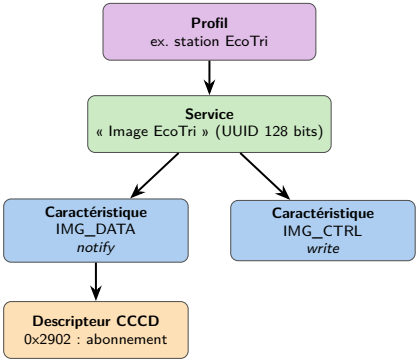
- Le **serveur GATT** possède les données (souvent le périphérique, ex. capteur).
- Le **client GATT** les lit, les écrit ou s'y abonne (souvent le central).
- GAP (qui se connecte) et GATT (qui possède les données) sont **indépendants** : la Portenta peut être centrale **et** cliente, ou centrale **et** serveur.

UUID

- **16 bits** : réservés au Bluetooth SIG (ex. 0x180F service Batterie, 0x2A19 niveau de batterie, 0x2902 CCCD).
- **128 bits** : services propriétaires, générés aléatoirement (`uuidgen`).
- Un UUID 16 bits est un raccourci de `0000xxxx-0000-1000-8000-00805F9B34FB`.

GATT et ATT

Profil, service, caractéristique, descripteur

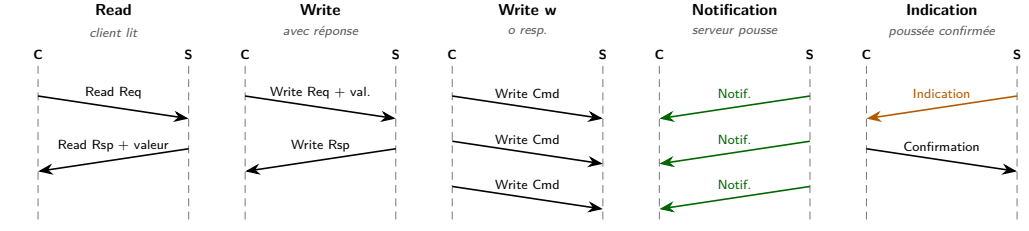


Handle	Type (UUID)	Valeur	Perm.
0x0010	0x2800 Service primaire	UUID du service	R
0x0011	0x2803 Décl. caract.	notify, h=0x0012, UUID	R
0x0012	UUID IMG_DATA	fragment ($\leq$ MTU$-$3 o)	–
0x0013	0x2902 CCCD	0x0001 = notif. ON	R/W
0x0014	0x2803 Décl. caract.	write, h=0x0015, UUID	R
0x0015	UUID IMG_CTRL	commande / NACK	W

Tout est « attribut » : un handle de 16 bits, un type (UUID), une valeur et des permissions.
Le client ne manipule que des handles ; les UUID ne servent qu'à la découverte.

GATT et ATT

Les opérations ATT



1 aller-retour par valeur ⇒ au plus 1 valeur par T_{CI} (souvent 2 T_{CI})

pas d'attente applicative ⇒ plusieurs valeurs par événement

1 confirmation / valeur

*Dans tous les cas la couche liaison acquitte chaque paquet (ACK + CRC) : ce qui manque à « notify », c'est l'accusé de réception **de l'application**.*

notify pour un flux du serveur vers le client (images, mesures) ; **write without response** pour un flux rapide du client vers le serveur ; **indicate** et **write** pour les rares messages qui doivent être confirmés par l'application.

- Une caractéristique *notify* ne notifie **rien** tant que le client n'a pas écrit dans son **CCCD** (*Client Characteristic Configuration Descriptor*, UUID 0x2902).
- Valeur 16 bits : bit 0 = notifications, bit 1 = indications.
- Le CCCD est propre à **chaque client** : le serveur doit mémoriser qui s'est abonné.

```
# Côté client (Portenta, aioble)
svc  = await conn.service(SVC_UUID)
data = await svc.characteristic(DATA_UUID)
await data.subscribe(notify=True)  # écrit 0x0001
while True:
    trame = await data.notified()
```

Erreur classique

« Mon ESP32 envoie mais je ne reçois rien » : le client ne s'est pas abonné, ou le serveur n'a pas déclaré de CCCD.

- **LE Secure Connections** (BLE 4.2+) : échange de clés ECDH P-256, puis lien chiffré **AES-CCM 128 bits** par la couche liaison.
- Méthodes d'appairage : *Just Works* (pas de protection MITM), *Passkey*, *Numeric Comparison*, *Out of Band* (NFC, QR).
- Une caractéristique peut exiger un lien **chiffré** ou **authentifié** dans ses permissions.

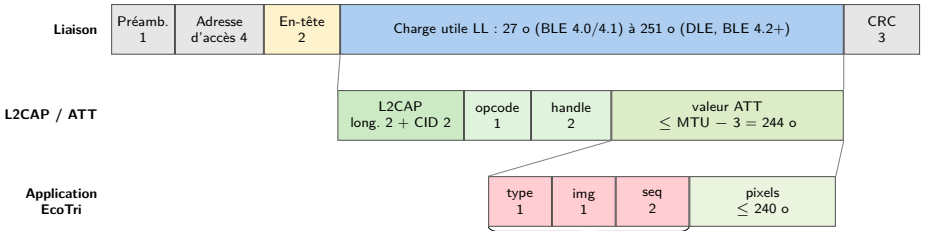
Risque pour EcoTri

Sans appairage, n'importe quel téléphone peut écrire `RESET` dans COMMANDE et arrêter le tri.

> **!** 4 octets de MIC par paquet chiffré et un appairage unique ; approfondi en lecture 7 (sécurité de l'Edge).

GATT et ATT

MTU, DLE et taille d'un paquet



Débit utile maximal : (MTU − 3 − 4) octets de pixels pour 261 octets émis sur le lien (couche applicative incluse), soit une efficacité de 240/261 ≈ 92 %. Sans DLE : 16/37 ≈ 43 %.

- **ATT MTU** : taille maximale d'un PDU ATT. 23 octets par défaut ⇒ 20 octets de valeur (23 − 1 opcode − 2 handle). Négociée par `Exchange MTU` : on garde le **minimum** des deux.
- **DLE** (*Data Length Extension*, BLE 4.2) : charge utile de la couche liaison 27 → 251 octets. Sans DLE, L2CAP découpe un grand PDU ATT en plusieurs paquets de liaison.

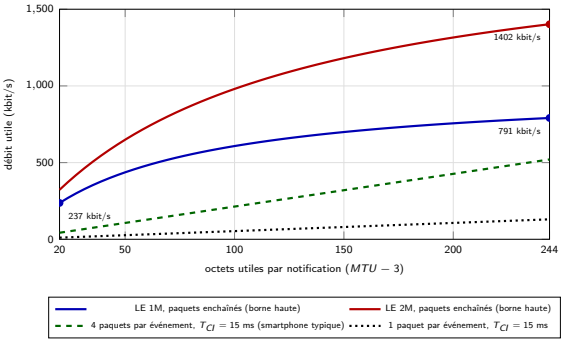
Modèle (paquets enchaînés)

Durée d'un échange donnée + ACK en LE 1M :

$$T_p = \underbrace{(L+10)\cdot 8\,\mu\text{s}}_{\text{paquet}} + 2\times 150\,\mu\text{s} + 80\,\mu\text{s}$$

$$D_{\text{utile}} = \frac{8\,(MTU-3)}{T_p}, \quad L = MTU + 4$$

Exemple : $MTU = 247 \Rightarrow T_p = 2{,}47$ ms, $D \approx$ **791 kbit/s**. En LE 2M : $\approx$ **1,4 Mbit/s**.



! la pile limite le nombre k de paquets par événement (téléphones : 4 à 7) : $D = k\,8\,(MTU-3)/T_{CI}$, on mesure plutôt **100 à 700 kbit/s**. Le calcul donne la **borne** ; la mesure du TP donne la **réalité**.

GATT et ATT

Le service EcoTri : premier jet du protocole (jalon J3)

Élément	UUID	Propriétés	Contenu
Service Image	`e0c00001-7b1a-4b2c-9d3e-ec0712a00000`	–	serveur : XIAO
IMG_DATA	`e0c00002-...`	notify	trames START / DATA / END
IMG_CTRL	`e0c00003-...`	write w/o resp.	commandes, ACK, NACK
Service Décision	`e0c00010-...`	–	serveur : Portenta
DECISION	`e0c00011-...`	read, notify	JSON de décision
COMMANDE	`e0c00012-...`	write	`LED`, `RESET`, `SEUIL`



une base 128 bits commune à l'équipe, et un compteur dans les octets 2 et 3 : on lit le rôle de chaque caractéristique dans l'UUID et les captures de nRF Connect restent lisibles.

Question flash

?

Avec un MTU négocié de 185 octets (valeur typique d'un iPhone), combien de notifications faut-il pour l'image de 9216 octets, avec un en-tête applicatif de 4 octets ? Et avec le MTU par défaut ?

Question flash

? Avec un MTU négocié de 185 octets (valeur typique d'un iPhone), combien de notifications faut-il pour l'image de 9216 octets, avec un en-tête applicatif de 4 octets ? Et avec le MTU par défaut ?

Réponse

Charge utile : $185 - 3 - 4 = 178$ octets $\Rightarrow \lceil 9216/178 \rceil =$ **52 notifications**. MTU par défaut : $20 - 4 = 16$ octets $\Rightarrow$ **576 notifications**, soit 11 fois plus de paquets, et autant d'en-têtes de liaison.

Fragmenter une image

Comment faire passer 9 Ko dans des paquets de 20 à 244 octets, sans rien perdre ?

Fragmenter une image

Pourquoi des pertes alors que la liaison acquitte tout ?

- La couche liaison BLE est **fiable** : CRC 24 bits, ACK et retransmission de chaque paquet tant que le lien vit.
- Les pertes surviennent **au-dessus** : file d'émission pleine (`notify` refusé), tampon du client écrasé avant lecture, déconnexion en cours de transfert.
- Sur MicroPython, une notification arrivée pendant que le script traite la précédente peut être **écrasée**.

Principe de bout en bout (Saltzer, Reed, Clark, MIT, 1984)

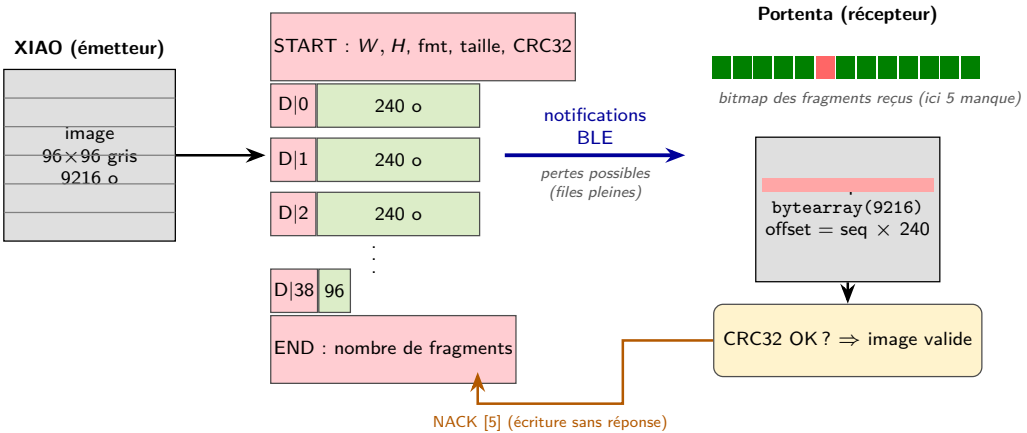
La fiabilité d'un transfert ne peut être garantie complètement que par les **extrémités** : les garanties des couches basses sont une optimisation, pas une preuve.



numéroter, vérifier l'intégrité de l'image entière, et redemander ce qui manque.

Fragmenter une image

Fragmentation indexée et reconstruction



Fragmenter une image

Le format des trames

Octets	Champ	Rôle
0	`type`	0x01 START, 0x02 DATA, 0x03 END
1	`img`	identifiant d'image (0–255, cyclique)
2–3	`seq`	n° de fragment, uint16 petit-boutiste
4–	`payload`	pixels, ou métadonnées (START)

Trame START (payload de 13 octets)

`W:u16 | H:u16 | fmt:u8 | taille:u32 | crc32:u32`

```python
import struct
HDR = "<BBH"                # type, img, seq
def trame(t, img, seq, payload=b""):
    return struct.pack(HDR, t, img, seq) + payload

start = trame(1, 7, 0,
    struct.pack("<HHBII", 96, 96, 0, 9216, crc))
```

! les deux MCU (Xtensa, Cortex-M) le sont : on lit l'entier sans permutation d'octets. Il faut l'**écrire** dans la spécification.

Fragmenter une image

- Un CRC est le reste de la division polynomiale du message par un polynôme générateur ; CRC-32 (IEEE 802.3) : $G(x) = x^{32} + x^{26} + \cdots + 1$.
- Il détecte **toutes** les erreurs de 1 ou 2 bits, toute rafale ≤ 32 bits, et une erreur aléatoire avec une probabilité $1 - 2^{-32}$.
- Il protège ici contre les **erreurs de reconstruction** (fragment mal placé, doublon, image mélangée), pas contre le bruit radio, déjà traité par la couche liaison.

```python
import binascii              # MicroPython / OpenMV
crc = binascii.crc32(buf) & 0xFFFFFFFF
```

```c
#include <esp_crc.h>        // XIAO (ESP-IDF)
uint32_t crc = esp_crc32_le(0, buf, len);
```

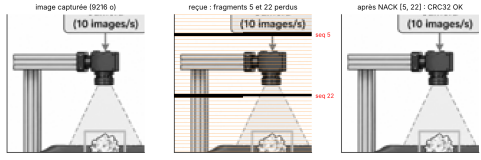
Piège

les deux bibliothèques doivent donner **le même nombre** : testez sur `b"123456789"` (valeur attendue 0xCBF43926).

Fragmenter une image

Fiabilité : ARQ sélectif plutôt qu'acquittement par fragment

- **Stop-and-wait** (un ACK par fragment) : simple, mais un fragment par aller-retour ⇒ débit divisé par le nombre de paquets par événement.
- **ARQ sélectif** : l'émetteur envoie tout ; le récepteur renvoie à la fin la **liste des manquants** (NACK) ; on ne retransmet que ceux-là.
- Temps total : $T \approx \frac{N_f}{r} + T_{Cl}\,(1 + k)$, avec r fragments/s et k tours de NACK.



Effet de deux fragments perdus (seq 5 et 22) : des bandes de 2,5 lignes manquent. Après un tour de NACK, le CRC32 valide l'image.

Question flash

?

On mesure un taux de perte de 3 % par fragment, indépendant, pour 39 fragments. Quelle est la probabilité de recevoir l'image entière du premier coup ? Combien de fragments à renvoyer en moyenne ?

Question flash

?

On mesure un taux de perte de 3 % par fragment, indépendant, pour 39 fragments. Quelle est la probabilité de recevoir l'image entière du premier coup ? Combien de fragments à renvoyer en moyenne ?

Réponse

$P = 0{,}97^{39} \approx \mathbf{0{,}30}$: 70 % des images seraient incomplètes sans retransmission ! En moyenne $39 \times 0{,}03 \approx \mathbf{1{,}2\ fragment}$ à renvoyer : le NACK sélectif coûte à peine 3 % de débit, alors que renvoyer toute l'image coûterait 100 % à chaque échec.

Protocoles et énergie

BLE, Wi-Fi, MQTT ou OPC UA : lequel, où, et à quel coût énergétique ?

Protocoles et énergie

Wi-Fi : débit élevé, coût fixe élevé

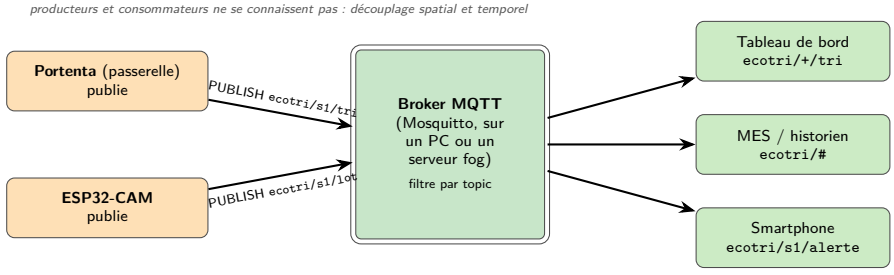
- IEEE 802.11 b/g/n (ESP32, Portenta) : 5 à 20 Mbit/s utiles en TCP.
- Courant en émission $\approx$ 200–300 mA ; **association** au point d'accès : 1 à 3 s.
- Énergie par bit **plus faible** que BLE pour les gros volumes (images, flux), mais coût fixe et veille bien plus élevés.
- Pile IP complète : HTTP, MQTT, OPC UA deviennent possibles.

	BLE	**Wi-Fi**
Débit utile	0,1–1,4 Mbit/s	5–20 Mbit/s
Connexion	$\approx$ 10 ms	1–3 s
Veille connectée	μA	mA
Topologie	étoile, point à point	étoile (AP)
IP natif	non	oui

! gros volume rare $\Rightarrow$ Wi-Fi ; petit volume fréquent sur batterie $\Rightarrow$ BLE.

Protocoles et énergie

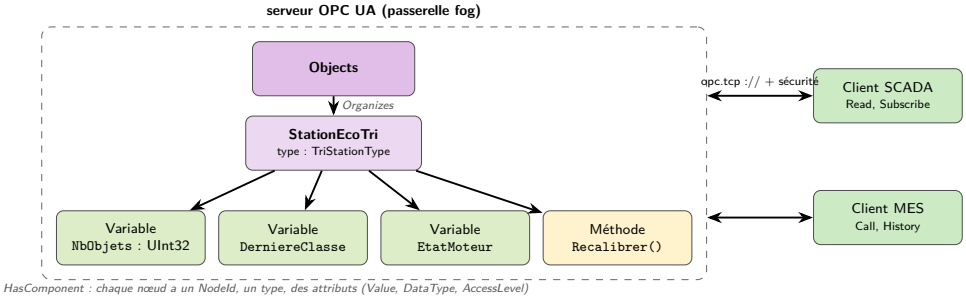
MQTT : publication / abonnement (OASIS, ISO/IEC 20922)



QoS 0 *: au plus une fois (aucun ACK)* · ***QoS 1*** *: au moins une fois (PUBACK, doublons possibles)* ·
QoS 2 *: exactement une fois (4 messages)* · ***retain*** *: dernier état conservé* · ***LWT*** *: message testament si le client disparaît*

- Protocole **applicatif** au-dessus de TCP (port 1883, 8883 en TLS) ; en-tête fixe de **2 octets**.
- Topics hiérarchiques ; jokers + (un niveau) et # (tous les niveaux).

```
from mqtt import MQTTClient       # OpenMV
c = MQTTClient("portenta", "192.168.1.10")
c.connect()
c.publish("ecotri/s1/tri", '{"cls":"metal"}')
```



- Plus qu'un transport : un **modèle d'information** typé, découvrable, avec méthodes, alarmes, historique et sécurité (certificats X.509).
- Lourd pour un MCU : il tourne sur la **passerelle fog** ; il existe un mode PubSub (sur MQTT ou UDP) pour les flux légers. Approfondi en lecture 7 (intégration MES).

Protocoles et énergie

Comparatif : à chaque lien son protocole

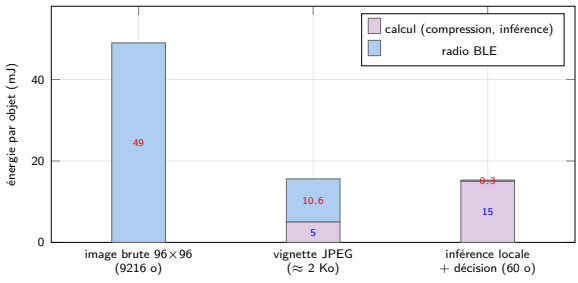
	BLE (GATT)	**Wi-Fi + HTTP**	**MQTT**	**OPC UA**
Couche	PHY à application	PHY + appli.	application (TCP)	application (TCP, UDP)
Modèle	client/serveur d'attributs	requête/réponse	publication/abonnement	client/serveur + PubSub
Charge utile typique	20–244 o	Ko à Mo	octets à Ko	Ko
Énergie	très faible	élevée	selon le lien	élevée
Interopérabilité	profils SIG	universelle	bonne (topics libres)	excellente (modèle typé)
Place dans EcoTri	XIAO ↔ Portenta, opérateur	flux de supervision	passerelle → fog	fog → MES

Ne pas comparer des pommes et des poires

BLE et Wi-Fi sont des **technologies radio** ; MQTT et OPC UA sont des **protocoles applicatifs** qui peuvent passer sur Wi-Fi, Ethernet… ou même BLE (via IPv6/6LoWPAN). Les comparer n'a de sens que pour un lien donné.

Protocoles et énergie

« Envoyer la décision, pas l'image » : le bilan sur la XIAO



Hypothèses : $E_b = 0{,}66\ \mu$J/bit en BLE ; compression JPEG 20 ms et inférence INT8 60 ms à 0,25 W ; capture identique dans les trois cas (non comptée).

! la décision locale divise par **3** l'énergie par rapport à l'image brute, mais n'est pas meilleure qu'un JPEG ici. Ses vrais gains sont ailleurs : **débit** libéré (150×), **confidentialité**, **passage à l'échelle** (10 nœuds sur un seul central).

JSON (lisible, 60–80 octets)

```
{"v":1,"st":"s1","id":412,"t":51234,
 "cls":"metal","p":0.93,"lot":"L-0412"}
```

Binaire compact (11 octets)

```
struct.pack("<BHIBBH", 1, 412, 51234,
            3, 93, 412)  # v,id,t,cls,p%,lot
```

- JSON : auto-descriptif, se lit dans nRF Connect et se publie tel quel sur MQTT ; mais **6 à 7 fois plus long**.
- Binaire : tient dans une notification de 20 octets et dans une publicité ; demande une **spécification** exacte.
- Compromis normalisé : **CBOR** (RFC 8949), JSON encodé en binaire.

> **!** choisissez un format, versionnez-le (champ `v`) et documentez chaque champ (type, unité, plage).

Question flash

?

La direction veut recevoir sur le MES, en temps réel, chaque image classée « inconnu » pour audit, en plus des statistiques. Proposez le chemin de chaque donnée (nœud, protocole, format) et estimez la charge si 5 % des 2 objets/s sont « inconnus ».

Question flash

?

La direction veut recevoir sur le MES, en temps réel, chaque image classée « inconnu » pour audit, en plus des statistiques. Proposez le chemin de chaque donnée (nœud, protocole, format) et estimez la charge si 5 % des 2 objets/s sont « inconnus ».

Réponse

XIAO → Portenta : image 96×96 en BLE fragmenté (déjà disponible). Portenta → broker : JPEG (≈ 2 Ko) en MQTT QoS 1 sur Wi-Fi, topic `ecotri/s1/audit`. Statistiques : MQTT retain, puis pont OPC UA vers le MES. Charge : $2 \times 0{,}05 = 0{,}1$ image/s $\times$ 2 Ko $\approx$ **1,6 kbit/s** : négligeable pour le Wi-Fi, et l'image ne quitte pas le site.

À retenir

- **Un protocole par lien** : commande, volume ou indicateur n'ont pas les mêmes exigences.
- **BLE** : GAP pour se trouver (publicité, connexion, T_{CI}) ; GATT pour échanger (services, caractéristiques, CCCD) ; ATT ne connaît que des handles ; SMP sécurise.
- **Débit** : $D = 8\,(MTU - 3)/T_p$ par paquet enchaîné ; négocier le MTU, activer DLE et 2M, enchaîner les notifications. Puis **mesurer**.
- **Fiabilité de bout en bout** : en-tête (type, image, seq), CRC32 sur l'image, NACK sélectif.
- **Wi-Fi, MQTT, OPC UA** : du gros volume au modèle d'information industriel, sur la passerelle.
- **Énergie** : un bit radio coûte 10^4 fois une opération ; envoyer la décision libère surtout le canal et protège la donnée.

Glossaire et références

Terme	Définition courte
GAP	découverte et connexion
GATT	organisation des données
ATT	protocole d'attributs (handles)
CCCD	descripteur d'abonnement 0x2902
MTU	taille max. d'un PDU ATT
DLE	charge liaison jusqu'à 251 o
T_{CI}	intervalle de connexion
SMP	appairage et clés
ARQ	retransmission automatique
QoS	niveau de garantie MQTT

- Bluetooth SIG, *Core Specification v5.4*, vol. 3 (Host) et vol. 6 (Low Energy Controller), 2023.
- K. Townsend, C. Cufí, Akiba, R. Davidson, *Getting Started with Bluetooth Low Energy*, O'Reilly, 2014.
- C. Gomez, J. Oller, J. Paradells (UPC Barcelona), « Overview and Evaluation of Bluetooth Low Energy », *Sensors*, 12(9), 2012.
- M. Siekkinen et al. (Aalto University), « How Low Energy is Bluetooth Low Energy ? », IEEE WCNC Workshops, 2012.
- A. Dementyev et al. (U. Washington, Microsoft Research), « Power Consumption Analysis of BLE, ZigBee and ANT », IEEE IWS, 2013.
- J. H. Saltzer, D. P. Reed, D. D. Clark (MIT), « End-to-End Arguments in System Design », *ACM TOCS*, 2(4), 1984.
- OASIS, *MQTT Version 5.0*, 2019 ; IEC 62541, *OPC Unified Architecture*.
- V. Janapa Reddi et al. (Harvard), *Machine Learning Systems*, mlsysbook.ai.
- Documentations : OpenMV `aioble`, Espressif ESP32-S3 BLE, Nordic SoftDevice (débits BLE).

Prochaine étape

Tutorial 3 : service BLE sur la Portenta avec `aioble`. **TP 3** : image fragmentée XIAO → Portenta, mesures de débit et jalon J3.